

Predição de Problemas Cardíacos por Modelos de Classificação de Aprendizado de Máquina

João Victor Lopez Pereira^{1,2} e Yuri Rocha de Albuquerque^{1,3}

¹ Universidade Federal do Rio de Janeiro, Rio de Janeiro RJ, Brasil

² joaovictorlopezpereira@gmail.com

³ yurira@ic.ufrj.br

Resumo O trabalho investiga o desempenho de três algoritmos de classificação — KNN, Perceptron e Regressão Logística — aplicados ao “Heart Failure Prediction Dataset”, que contém informações clínicas de 918 pacientes com ou sem doença cardíaca. Após um pré-processamento que envolveu limpeza de dados, remoção de atributos redundantes e codificação *one-hot* de variáveis categóricas, foram avaliadas as correlações entre as *features* e realizada uma curadoria final do conjunto utilizado nos experimentos. Os métodos foram treinados usando validação cruzada com 10 folds, repetida 101 vezes para garantir variabilidade estatística. Em cada execução, foram registradas acurácia e *f1-score*. Os resultados médios indicaram desempenho superior da Regressão Logística (acurácia 0.867; $F1 = 0.861$), seguida do KNN e, por último, do Perceptron. A análise dos histogramas mostrou que as distribuições das métricas se aproximam de normais, mas com variâncias distintas: a Regressão Logística exibiu a menor variabilidade, indicando maior estabilidade frente às mudanças nos folds, enquanto o Perceptron apresentou forte sensibilidade aos dados. Testes de hipótese de Friedman e Nemenyi como *Post hoc* confirmaram estatisticamente as diferenças observadas, rejeitando as hipóteses nulas de desempenho equivalente entre os modelos. Conclui-se que, para essa base de dados e parametrizações escolhidas, a Regressão Logística foi o método mais robusto e consistente.

Palavras-chave: Aprendizado de Máquina · KNN · Regressão Logística · Perceptron.

1 Introdução

O objetivo desse trabalho é comparar modelos de inteligência artificial e descobrir qual deles possui a maior performance quando treinados em cima de um *dataset* que contém dados de diversos pacientes que possuem, ou não, uma doença cardiovascular. Escolhemos esse tema pois, de acordo com a OMS, doenças cardíacas são a principal causa de morte no mundo, logo, queríamos utilizar o que foi aprendido para aplicar em um problema muito impactante globalmente na atualidade.

2 Descrição da Base e Preprocessamento

A base de dados selecionada, chamada “Heart Failure Prediction Dataset”, contém 918 observações com 12 atributos cada a respeito de pacientes que contém ou não alguma doença cardíaca. Segundo a descrição do próprio repositório no portal *Kaggle*, onde a base de dados está disponível, problemas cardíacos são a principal causa de morte no mundo, com uma estimativa de 17.9 milhões de mortes todos os anos.[1]

“People with cardiovascular disease or who are at high cardiovascular risk (due to the presence of one or more risk factors such as hypertension, diabetes, hyperlipidaemia or already established disease) need early detection and management wherein a machine learning model can be of great help.”

Fedesoriano, *Heart Failure Prediction Dataset*. [1]

Os atributos contidos na base de dados são:

1. **Age**: Idade do paciente (Valor numérico);
2. **Sex**: Gênero do paciente (M: Male, F: Female);
3. **ChestPainType**: Tipo de dor no peito que o paciente sofre (TA: Typical Angina, ATA: Atypical Angina, NAP: Non-Anginal Pain, ASY: Asymptomatic);
4. **RestingBP**: Pressão arterial em repouso (Valor numérico);
5. **Cholesterol**: Colesterol do paciente (Valor numérico);
6. **FastingBS**: Alto (1) ou baixo (0) nível de glucose no sangue em jejum (Valor booleano);
7. **RestingECG**: Resultado do eletrocardiograma em repouso (Normal: Normal, ST: having ST-T wave abnormality (T wave inversions and/or ST elevation or depression of > 0.05 mV), LVH: showing probable or definite left ventricular hypertrophy by Estes’ criteria);
8. **MaxHR**: Maior valor de taxa de batimento cardíaco atingido (Valor numérico);
9. **ExerciseAngina**: Se o paciente sente (Y) ou não (N) dor no peito ao realizar exercício (Y: Yes, N: No);
10. **Oldpeak**: Rebaixamento do segmento ST observada em um eletrocardiograma (Valor numérico);
11. **ST_Slope**: Comportamento de inclinação do segmento ST observada em um eletrocardiograma (Up: upsloping, Flat: flat, Down: downsloping);
12. **HeartDisease**: Índica presença (1) ou ausência (0) de doença cardíaca (Valor booleano) (nossa variável *target*).

Primeiro, optamos por decompor os atributos **ChestPainType**, **RestingECG** e **ST_Slope** em variáveis booleanas, permitindo seu uso adequado nos modelos de classificação. Essa transformação foi feita por meio do *One-hot Encoding*, de forma que os dados categóricos fossem convertidos em formato numérico

sem introduzir viés algorítmico decorrente de codificações inadequadas, como a simples atribuição de valores inteiros sequenciais (por exemplo, de 1 a 4 em `ChestPainType`). Além disso, convertemos os atributos de classificação binária `Sex` e `ExerciseAngina` em valores numéricos 0 e 1 pelos mesmos motivos.

Em seguida, verificamos a existência de espaços vazios, duplicatas ou valores indevidos no conjunto de dados: não foram encontrados espaços vazios e nem duplicatas, mas em 2 variáveis (`RestingBP` e `Cholesterol`) foram encontrados entradas nulas que, naturalmente, não fazem sentido. Decidimos, portanto, não trabalhar com essas específicas observações. Em seguida, visualizamos o histograma de cada variável — tanto numérica quanto booleana — e vimos que, no geral, os dados parecem estar razoavelmente bem distribuídos, com exceção daqueles resultantes do processo de *One-hot Encoding*. Além disso, foi observado que várias das variáveis numéricas, como `age`, `RestingBP`, `Cholesterol` e `MaxHR`, apresentam distribuições próximas de uma gaussiana. A variável numérica `Oldpeak`, em contraste com as outras, apresenta uma distribuição semelhante à uma exponencial, porém, discretizada.

Após essa etapa de préprocessamento inicial, foram montadas as matrizes de correlação para a análise da linearidade entre as próprias *features* (variáveis independentes) e entre as *features* e o *output* (variável dependente).

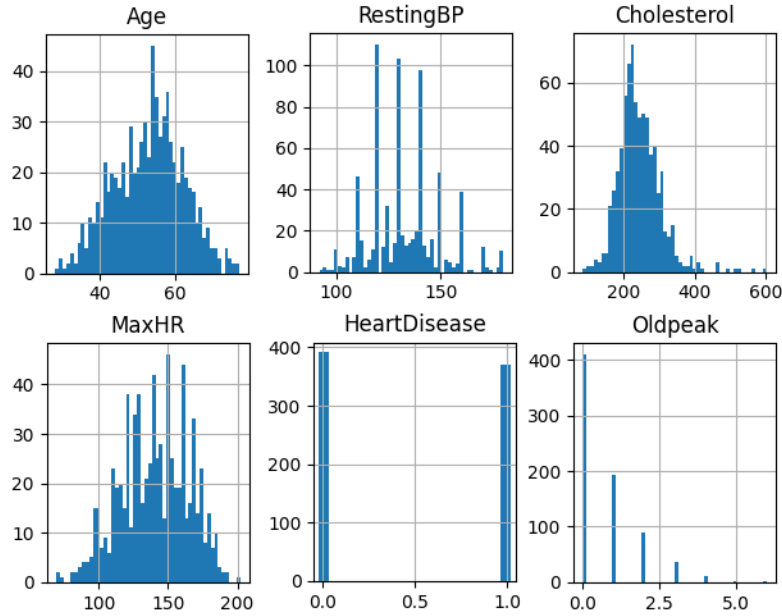


Figura 1. Histograma das *features* não binárias do *dataset*, com exceção da variável *target*.

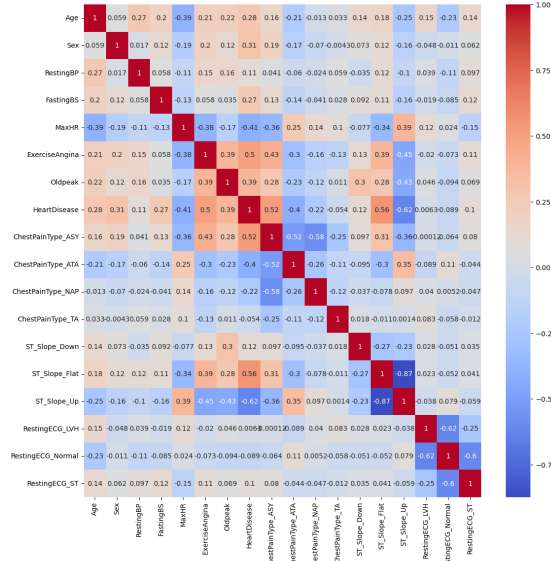


Figura 2. Mapa de calor da matriz de correlação de Pearson.

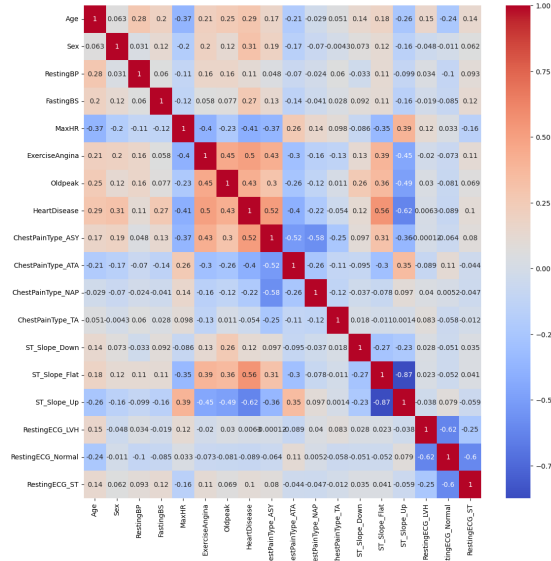


Figura 3. Mapa de calor da matriz de correlação de Spearman.

Pela análise das correlações, percebe-se que as *features* `ST_Slope_Up` e `ST_Slope_Flat` possuem forte correlação entre si e, por serem variáveis independentes, é indesejado esse comportamento. Portanto, a segunda foi removida do *dataset* final. Outra análise válida é a baixíssima correlação entre as *features* `RestingECG_LVH` e `RestingECG_Normal` com a variável dependente, o que também é indesejável. Porém, nesse caso, não se sabe se é porque, de fato, a relação linear é baixa ou não é uma relação linear (mesmo utilizando o método de Spearman, ainda não se pode ter total certeza). Ainda assim, decidimos por eliminar ambas as *features* da base de dados.

3 Definição dos Métodos Usados e Justificativa para sua Utilização

Utilizamos, ao todo, três métodos:

1. KNN (*K-Nearest Neighbors*): Método baseado em similaridade: para classificar um ponto, olha os k vizinhos mais próximos e classifica o ponto de acordo com a classe da maioria. Não aprende um modelo; apenas armazena os dados.
2. Perceptron: Modelo linear que ajusta um hiperplano separador atualizando pesos a partir de erros de classificação. Funciona apenas quando os dados são linearmente separáveis.
3. Regressão Logística: Modelo que estima a probabilidade de uma classe usando a função sigmoide. No nosso caso, foi usado o modelo de classificação binária.

Escolhemos esses métodos pois, além dos três serem algoritmos de classificação (objetivo do trabalho), são simples o suficiente para entendermos o funcionamento e foram dados inteiramente na disciplina.

4 Trabalhos Relacionados

Após uma breve pesquisa por trabalhos relacionados que se utilizam da mesma base de dados que nós utilizamos, encontramos algumas postagens em portais *online* que obtiveram acurácia e **f1-score** bem semelhante ao nosso, mas nada que fosse consideravelmente melhor ou pior.

Algo que percebemos foi que grande parte dessas postagens usaram diversos modelos — assim como nós fizemos — para obter seus resultados e julgar qual deles foi o mais robusto. Vários deles usaram modelos que não conhecemos o suficiente, como o *Gradient Boosting Classifier*, *SVM* e *Random Forest*.

5 Experimentos Realizados

Separamos o *dataset* em 10 *folds* e rodamos 101 vezes os mesmos 10 *folds* para os três modelos, aleatorizando a ordem dos registros da tabela em cada uma das

101 iterações, guardando a média da acurácia e do **f1-score** de cada *fold* de cada iteração em um vetor. Após o treinamento, utilizamos testes de hipótese e plotamos gráficos de *boxplot* para avaliar a robustez dos modelos entre si. Para o teste de hipótese, realizamos o Teste de Friedman visto que tínhamos três modelos e os dados eram pareados (pois estamos utilizando o mesmo *fold* para o treinamento de cada modelo para evitar a criação de viés). Após esse teste, realizamos como *Post hoc* o teste de Nemenyi para avaliar, par a par, se os resultados dos testes eram estatisticamente equivalentes ou não.

Além disso, para todos os experimentos, usamos o KNN com um total de 7, 9 e 11 vizinhos, a Regressão Logística com um total de 5000, 6000 e 7000 iterações e o Perceptron com um total de 2000, 3000 e 4000 iterações e 10^{-3} de tolerância. Após realizar os testes com as variações nos argumentos de cada algoritmo, observamos que não houve mudança significativa nos resultados alcançados, com isso, decidimos por utilizar o KNN com 7 vizinhos, a Regressão Logística com 5000 iterações e o Perceptron com 2000 iterações.

6 Discussão dos Resultados

Os resultados que obtivemos foram, em média, após 101 iterações:

1. Acurácia média do Perceptron: 0.797
2. Acurácia média do KNN: 0.851
3. Acurácia média da Regressão Logística: 0.867

1. **F1-score** médio do Perceptron: 0.778
2. **F1-score** médio do KNN: 0.845
3. **F1-score** médio da Regressão Logística: 0.861

Além disso, ao observar o histograma dos resultados alcançados, percebe-se que as métricas obtidas apresentam distribuição semelhante à uma curva normal, com o porém da variância obtida para cada uma delas ter sido diferente. A variância da Regressão Logística, em comparação com as demais, foi relativamente pequena, o que indica que o método não se mostrou sensível a depender do *fold* onde os treinos e testes foram realizados. Além disso, pela função de log-verossimilhança da Regressão Logística ser suave e apresentar ponto de máximo, métodos numéricos, como o gradiente descendente, encontram uma excelente aproximação pro seu ponto de máximo global, resultando em uma fronteira de decisão estável. Em contrapartida, as métricas do perceptron apresentaram variância relativamente alta em comparação com às métricas dos outros métodos, visto que o perceptron, que não funciona bem com dados não-linearmente separáveis, se mostrou bastante sensível aos dados de treino e teste.

Além disso, as métricas do KNN foram inferiores às métricas da Regressão Logística pois o algoritmo do KNN classifica apenas de acordo com a distância euclidiana dos dados, sem considerar diferentes níveis de importância entre diferentes *features*, ou seja, se há uma relação linear monotônica entre as variáveis,

essa relação é facilmente captada pela regressão logística, ao passo que não é captada pelo KNN.

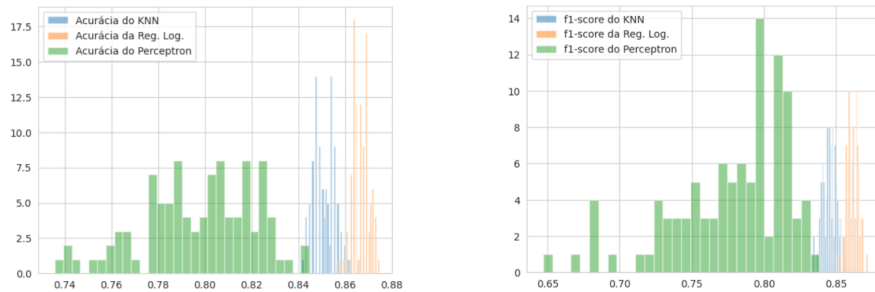


Figura 4. Histograma da acurácia e f1-score obtido nos testes realizados.

Por fim, rodamos o teste de hipótese de Friedman para garantir estatisticamente que, de fato, os modelos apresentam resultados diferentes. Após isso, realizamos como teste de *Post hoc* o teste de Nemenyi para visualizar quais dos modelos diferem entre si.

```
>> _, p_value = friedmanchisquare(knn_acc,
                                   reglog_acc,
                                   perceptron_acc)

>> print(p_value)
1.3685394711738522e-44

>> data = np.column_stack([knn_acc,
                             reglog_acc,
                             perceptron_acc])

>> posthoc = sp.posthoc_nemenyi_friedman(data)
>> print(posthoc)
```

	0	1	2
0	1.000000e+00	3.574696e-12	3.574696e-12
1	3.574696e-12	1.000000e+00	0.000000e+00
2	3.574696e-12	0.000000e+00	1.000000e+00

onde:

- `knn_acc` representa o vetor com as acurácias medidas do KNN;
- `perceptron_acc` representa o vetor com as acurácias medidas do Perceptron;
- `reglog_acc` representa o vetor com as acurácias medidas do Regressão Logística.

Como esperado, obtivemos que todos os modelos apresentam resultados estatisticamente diferentes entre si. Ou seja, como podemos observar analisando as entradas da matriz, todos os pares de modelos apresentaram resultados diferentes, o que indica que, para todo par, existe um modelo que apresentou resultado mais robusto. Logo, existe um modelo mais robusto frente a todos os outros.

Com base nos resultados, observando as médias e distribuições das acurácias e **f1-score** obtidos, podemos concluir que, de fato, o modelo de Regressão Logística, para esse teste, com os parâmetros e *features* selecionados, foi o mais robusto dentre os modelos testados.

7 Conclusão

Evidencia-se que a testagem de diferentes modelos ao atacar uma base de dados específica é de extrema importância, dado que diferentes modelos apresentam diferentes cenários onde seu funcionamento será adequado ou não. Além disso, acreditamos que o resultado encontrado tenha sido extremamente satisfatório e coerente com o que esperávamos.

Declaração de Conflito de Interesses: Os autores declaram que não possuem conflitos de interesse relevantes para o conteúdo deste artigo.

Referências

1. Fedesoriano: Heart Failure Prediction Dataset. <https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction>, last accessed 2025/11/11
2. Pereira, J.V.L., Albuquerque, Y.R.: Heart-Failure-Predictor: Machine learning project for heart failure prediction using clinical data. <https://github.com/joavictorlopezpereira/Heart-Failure-Predictor>, last accessed 2025/11/12